Changing Arrays

The Ruby array has lots of options for assigning new values or changing the array. As with strings, the []= method replaces whatever subarray would be returned by the square bracket lookup with the right side of the assignment, even if the two parts are different lengths.

Ruby provides multiple methods for adding and removing elements from an array that are specialized based on where in the array the change will be made:

- The beginning of the array—Items can be added to the beginning of the array using the prepend method (aliased as unshift), which takes multiple arguments and adds them to the front of the array. The method shift returns the first element of the array and removes it from the array. An optional argument allows you to return and remove an arbitrary number of elements.
- The end of the array—Items can be added to the end of the array with the << operator, which takes one argument, or the append method (aliased as push), which takes multiple arguments and adds them one by one to the end of the array. To retrieve the last element of the array, the method pop returns the last element and removes it from the array. An optional argument allows you to return and remove an arbitrary number of elements.
- An arbitrary location in the array—Items can be added at an arbitrary location with insert. The first argument to insert is the index, and then any additional arguments are are added to the array one by one at that position. To remove an argument at a specific index, delete_at takes the index, returns the element at that index, and removes it from the array. The method delete takes an object argument and removes that argument from the array any time it appears.

The combination of push and pop allows you to treat an array as a last-in-first-out stack, whereas the combination of push and shift gives you a first-in-first-out queue.

The rotate method, called with no arguments, returns a new array where the first element of the original array is now the last element of the new array. With a positive integer argument, it performs that amount of rotations. If the argument is greater than the size of the array, the effective final number of elements rotated is argument % array.size. If the argument is negative, the rotation reverses—the end moves to the beginning:

```
x = [1, 2, 3, 4, 5]
x.rotate     # => [2, 3, 4, 5, 1]
x.rotate(2)  # => [3, 4, 5, 1, 2]
x.rotate(-1) # => [5, 1, 2, 3, 4]
x.rotate(-2) # => [4, 5, 1, 2, 3]
```

The flatten method returns a new array that converts the original array to a one-dimensional array. That is, any subarray is replaced by inserting its original elements:

```
[1, [2, 3], 4, [5, 6, [7, 8]]].flatten # => [1, 2, 3, 4, 5, 6, 7, 8]
```

A convenient way to convert an array to a string is join, which takes a string delimiter and returns a string by converting each element of the array using to_s (recursively flattening if the element is itself an array). If no argument is given for the delimiter, the default is an empty string (well, the default is technically the global field separator value in $,, so you can change it if you wanted to):

```
[1, [2, 3], 4, [5, 6, [7, 8]]].join("|") # => "1|2|3|4|5|6|7|8"
```